

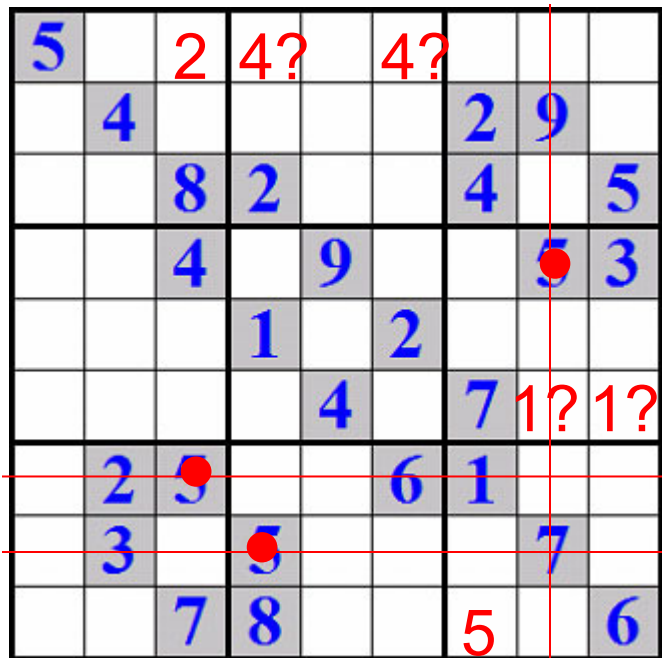
Mathematical Logic

SAT encoding

Puzzel: SUDOKU, again

Find placing *each number (1...9) s.t. precisely once in*

- *each row,*
- *each column,*
- *each 3*3 sub-square.*



```
MeadowNT.exe@MIRANDA
Buffers  Files  Tools  Edit  Search  Mule  Help
115
224
272
289
338
342
374
395
434
459
485
493
541
562
654
677
722
735
766
771
823
845
887
937
948
975
996
[--]-(Unix)--  sudokuMar2
```

SUDOKU as a SAT problem, again

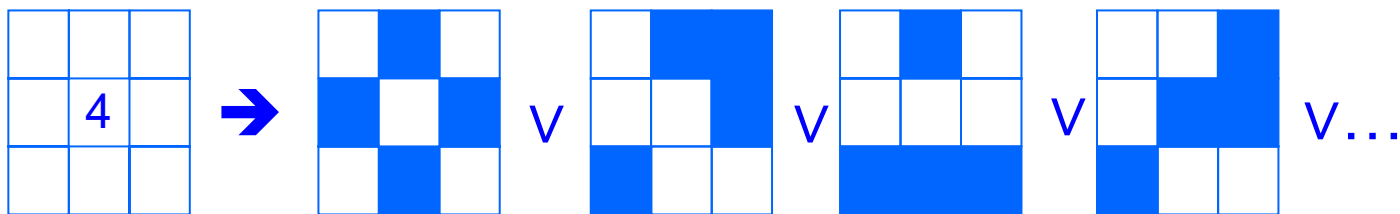
Lynce, I., Ouakline, J., AI Math 2006

- x_{ijk} iff k at (i,j) where $1 \leq i, j, k \leq 9$
- Each box contains precisely one number.
 $(x_{ij1} \vee x_{ij2} \vee \dots \vee x_{ij9}) \wedge (x_{ij1} \supset \neg x_{ij2})$ (i.e., $\neg x_{ij1} \vee \neg x_{ij2}$), ...
- Precisely once in each row (fix i):
 $(x_{i1k} \vee x_{i2k} \vee \dots \vee x_{i9k}) \wedge (x_{i1k} \supset \neg x_{i2k})$ (i.e., $\neg x_{i1k} \vee \neg x_{i2k}$), ...
- Precisely once in each column (fix j):
 $(x_{1jk} \vee x_{2jk} \vee \dots \vee x_{9jk}) \wedge (x_{1jk} \supset \neg x_{2jk})$ (i.e., $\neg x_{1jk} \vee \neg x_{2jk}$), ...
- Precisely once in each sub-3x3 matrix.
 $(x_{11k} \vee x_{12k} \vee \dots \vee x_{33k}) \wedge (x_{11k} \supset \neg x_{12k})$ (i.e., $\neg x_{11k} \vee \neg x_{12k}$), ...

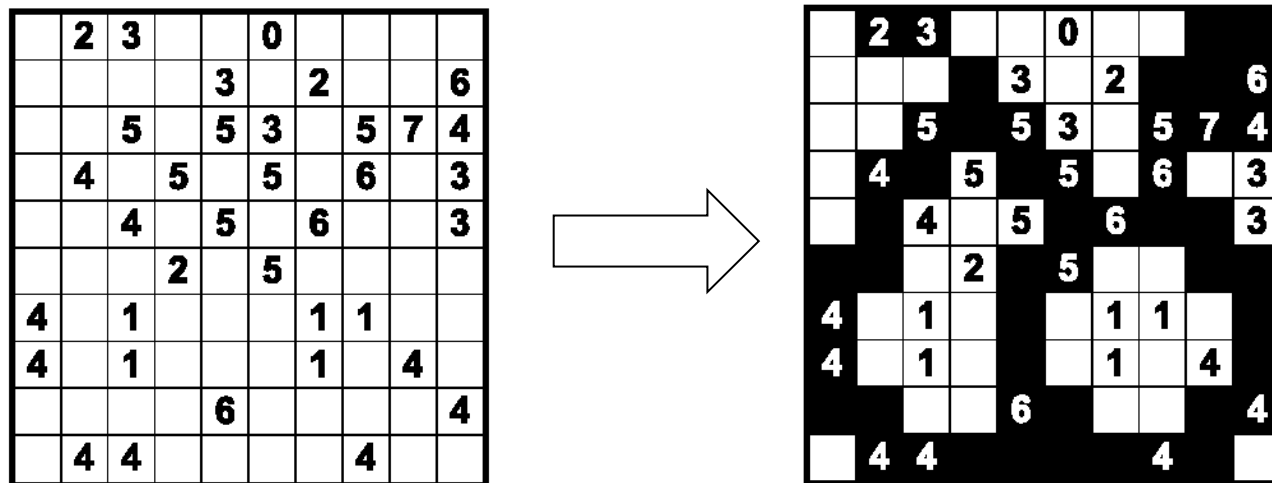
729vars, approx.8200clauses, in 10-50ms by miniSAT

Another puzzle

- *Problem:* Solve the puzzle of the next slide by Implementing a puzzle solver using miniSAT.
- ✓ *Rule of puzzle:* the number in a box is the number of black boxes among adjacent to it or itself, e.g.,



Ex.



Another puzzle

			1	2			1	4			4	6	4		4	5	4	3		3	2		1	0	3			3	3		3	1	4		0		3	2		1	2										
	5		5		2	2		2		3				3		2	3	1	3		4	2	2	3		1	2	3		2	3	3	2		3		3	3		1	3		2	2	3	4		2			
3		2	3	4	3	3	2	2	3		2	2		5	3		1	4			4				4	3	4	3		3	1	2	3		1		3		3		2	2	3	4		3					
3			3	2	1	3	2	2	3		3			4	3		3	5	2	4		3			4	3	3	3		3		4	3	2		3	2		5	2	3		5	2	3		4				
	5	5	2		1	3	2	3	3		3		4		0		5	4			3			3	3	2			5	3			6	5		3		3		5	2	3		5		4					
	7			3	2			4		4	6			5			6	5	6	7	3		1					5		4		6	4	4	4	5	4	6	5	3	3	4	3	2		7	5				
	5	5	2		1	1	3		8	6	6		4	6	7	5	3	8			1		3	7		7	3	3		7	7	5		7				9	6		3	2	2		5		3				
3	2	3			1			9	8		9			6	9	8			9			9		8	6		9	7	8		9	7		6	9		6	9		7	8		6	9		8	3	3	3	2	3
	5				3	4	9			8				6	8	7	6	7	8	8		8		9		9	7		9	8		8	5		9		8		9	9	7		2	3				2	3		
	5	6	3		1		8	6		4	8		7	6					8	9	9			9	9		6				5		5	6		9	7			9	7		2	3	6		2				
	6				6	6		4		7			1						6	6			6	9			8	6				2		7		2		5	5	2		4	6				2				
	3	3	0	4			5	5						2		1	1	2			2		3			6		7	6		6	6		5	5		4		5		3	3			2			2			
4			3					4		6		3	3	0	1	2	5	2			1	2					1	0		2		6	4	6	9	8			5	5			4	3	3				2		
	5	4		3			3	2	3	3	3			3	6		3	2	3	1		8		1	2		0		3	3		7	8	6			4	4		4		3	3		4	5	4	2			
		4	4			5	4		3	2	2		2	1	2	4		6	2	1		3	2		3	1		2			0		6	7		6	4	3	4		4	5	5	4	4						
1	2	3			2	4		3	2				1	1			4	6	6	6				1	3	3	3	2				6			5	5	3	3		6	4	2	3		4	2				2	
	2	2	3	4			5	3	3				2	2	3		2	5	2	2		2	2	2	2	2	3	2	1	3			7	7	5	5		5	3		4	4	4		4	2	2	2			
3	1	2	2		3		1	1		1	0		4	3	2			2	4	2	1		3			3		6		8	9	6			7	7	5	5		3	2	3	3		2	1		3			
4	4	2	1	2						4	3	3		1	1		0		5	5	3	2	3		3	3	4	4		9	6			7	8		6	5		1		3		3	1	2		3			
5	3	1			0	1	3		2	5			3			4	3		1	3	5	4	2			3		3	4	2	3	5			6	8	7	6		6	9		0	0		1	1	3		3	
5			2		3	4		5	7	4	3	3	3	3	3	3	3	3	3	3	2	5	5	3	2		3	2	2	2	2			8			9	8	5	3		0		2		3	3				
4	5		0	1		4		4	3	2	3	5	6		6	6	5	4		2	4	3	2		1	1	2		0	1		6	6	7	9			8	7		0	1		0	2		5	3			
	3		0	2			4		3		4	4	4	3		3	4	4	4		2	3	4	2			1		1	0	0	2		5		9	6		6		5	3		2			5	5	4		
	3	2	1	1		1		1	2	3	3	4	6	4	4	6	4	4	6	4	4	3	2		1	1	0		1	3		7	8	9			6	7		4	5	3	3	2			6	4			
1	2				1	3	3			4	2	2		5	4	4	4	4	2	1			2			1	3	3		7		9	6		7			4	4	4	4	3	1	1	3		4				
	2	1		2			4	3	4	4	3		3	5	3	3	3	3	3	3	2	4	4	1		1	2	2		2	3	1	1	2			7	9			5	2		4	5		2		2	2	
2	1		2	1			4		5	3	3	3		4	4	3			5	4		1	0	1	1			3		3			1	6		7	6			5	2		4	5		2		1	2	2	
3	2	2		0		3		2	5			3		3		3	0		2	6			1	0				1	2	2			7	9			7	6		2		5	4	2			1				
	2	1		5		3		5			3	3	4	5	5	4		3		6	3	1	0	3	4	2	1		3	2			6		6		9	6	6	6		6	0		3	2	3		0	1	
3	2		3		4	4	5	4	5		3	3	3		4		3		4	4		3			1	2		3	3			6		6		9	6	6	6		6		0		1	1	2	3	3	2	
	2	2	2		3	5	7	6	4	4	2			3		6	6	6					3		5		6	5		7		6	6	6	9			6		0		3	3	4		5	5	4			
2		6		3	4	5	6			1	0	1	3	3			3	3	3		5	6	4	2		2		5	6			6		7			6		4	5	3	2		2		0		5	3	1	
	4	5	4	2		4		5		2	3	2	3	3	3	2		3				6		5		5	6	5		5	6	5		5	5			5	6		5	4	3	3	4			4	5	5	4
	4	5			3		2					3	3	3		3	3		4	5		6	5	3		3	6		6	6			5	6			5	6		5	6		5	4			7	7	5	3	
7	3	3	4			3			6	4		3	3	4	4		5	5	5	4	0		5		4	6			2		2	2	4	4		5	4			4	5	5		1	2		2				
	6		3					8		5	5		4		6		3									6			6		5		5	6			6		5	6		6			3	3	4		3		3
6	6		6		6	8	9	8	6		6	8		8		4	3	1		3		3	4			4		4	3	4		4	6	5		4	6	5		3		6		3	3	4		3			3
2		6	4					6	7	7	8	6	6				1	1	3	3	4		7	7	5		6		6	6		5		6			5		6		8		2		5		2		3		
2	4	6	7		5	5		7			6	5			2	3	0	1	2	3			3	3		3	3	3		2	2		3	3	1		1		2		8		2		5		2		3		
4	3	6	6	4		6	3			6	6	8	7				0	2	2					5	3		6	3	6			2	1	3	3		3	5	6		6	2		7							
	4	4	4	7		3	5		2	3	4	5	7	7			8	8	5			3	3	3	3		3	2	5			3	3			2			6	3	3		1								
4	4	3	4	6		6		3	3		5		5		7	8	9	8	7		4	5		5	6		6			7		5	5	8			6		6		8	7		6		8	7				
2	4	4	4	3		6		2		3	4				5	5	6		8	6	4	5			6		6			5	6	5	7		4		7		6		6		8	7		6		8	7		
	3	4	5		6	6	3		3		4	4			4		4	3		6				6	7			6	7			7	6		6	8		7		6	6		6		6		6				
3	3	5	6		6	6					4	4	3	2			7	5	3	3	4	2	4			5		6	8			4	7	5			3	5			3		5		3	4	4	3			
2	3	3	4	6	6		6	6	3	4	5	4	5			5	4	4	4		6	6	7		6		3	2			3	2	3		8		5	8			3		4		5						
	3						4				4				6		5		2	3	4																														

Recall: Encoding exercise

- Describe the following by CNF
 - ✓ Among 4 logical variables, exactly 2 of them is true.
- First approach Length of CNF becomes $4^6 = 4096$
 - ~~✓ $(x_1 \wedge x_2 \wedge \neg x_3 \wedge \neg x_4) \vee (x_1 \wedge \neg x_2 \wedge x_3 \wedge \neg x_4) \vee \dots \vee (\neg x_1 \wedge x_2 \wedge \neg x_3 \wedge x_4) \vee (\neg x_1 \wedge \neg x_2 \wedge x_3 \wedge x_4)$~~
 - ~~✓ DNF (with length 6) and convert it to CNF.~~
- Second approach
 - ✓ Exactly 2 variables $\Leftrightarrow$ not “0 or 1 or 3 or 4 variables”.
 - ✓ Negation makes it to a CNF (length $1+4+4+1=10$).

Mathematical Logic
(5) Practical SAT encoding
Large Logic Picture

More on encoding
~Describing arithmetic~

CNF encoding of addition

x=3 (00011)	*	*			
y=6 (00110)		*	*		
z=9 (01001)	*			*	
c (01100)	/		*	*	
	0	1	2	3	4

3 "+" 6 = 9

auxiliary predicates

$$x \text{ xor } y = (x \wedge \neg y) \vee (\neg x \wedge y) = (x \vee y) \wedge (\neg x \vee \neg y)$$

x xor y xor z

$$= (x \wedge \neg y \wedge \neg z) \vee (\neg x \wedge y \wedge \neg z) \vee (\neg x \wedge \neg y \wedge z) \vee (x \wedge y \wedge z)$$

$$= (x \vee \neg y \vee \neg z) \wedge (\neg x \vee y \vee \neg z) \wedge (\neg x \vee \neg y \vee z) \wedge (x \vee y \vee z)$$

at_least_two (x,y,z)

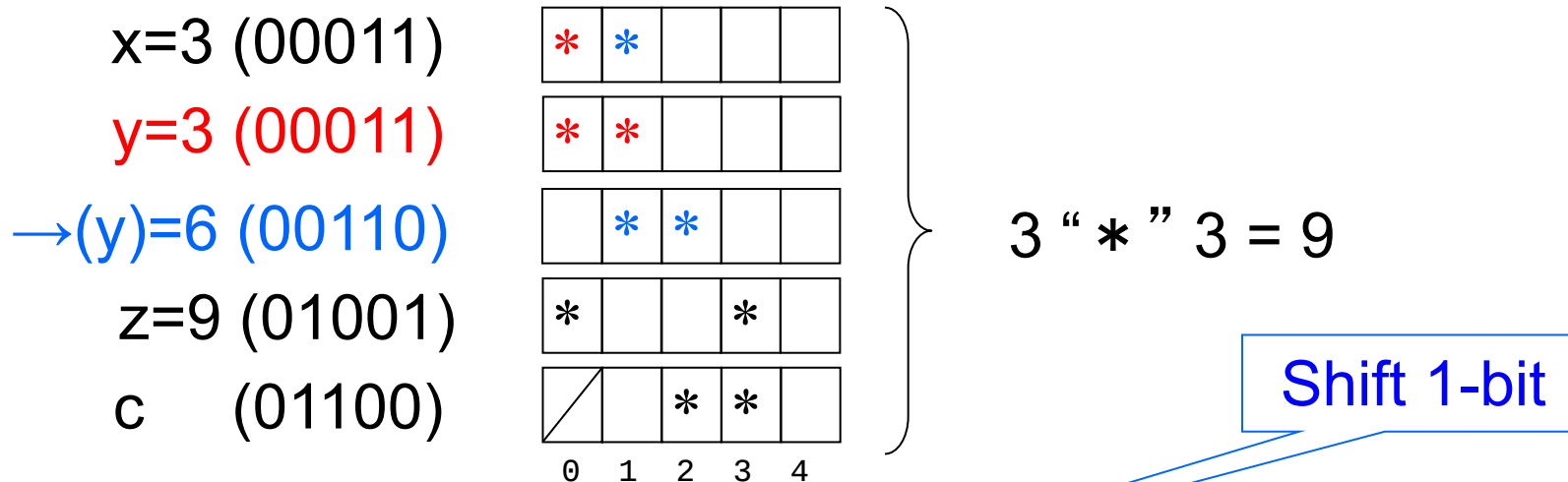
$$= (x \wedge y) \vee (x \wedge z) \vee (y \wedge z) = (x \vee y) \wedge (x \vee z) \wedge (y \vee z)$$

$\mathbf{x} \text{ "+" } \mathbf{y} = \mathbf{z}$ where $\mathbf{x}, \mathbf{y}$: n-bits, $\mathbf{z}$: n+1 bits, carry bits $\mathbf{c}$: n-bits

$$z_0 \Leftrightarrow x_0 \text{ xor } y_0, \quad z_i \Leftrightarrow x_i \text{ xor } y_i \text{ xor } c_{i-1} \quad (0 < i \leq n), \quad z_{n+1} \Leftrightarrow c_n$$

$$c_0 \Leftrightarrow x_0 \wedge y_0, \quad c_i \Leftrightarrow \text{at_least_two}(x_i, y_i, c_{i-1}) \quad (0 < i \leq n)$$

CNF encoding of multiplication



auxiliary predicates $\mathbf{x, y: n\text{-bits}, z: 2n\text{-bits}}$

For $x = x_0 x_1 x_2 \dots x_n$, $\rightarrow(x) = x'_0 x'_1 x'_2 \dots x'_n x'_{n+1}$

$\neg x'_0, x'_{i+1} \Leftrightarrow x_i \quad (0 \leq i \leq n)$

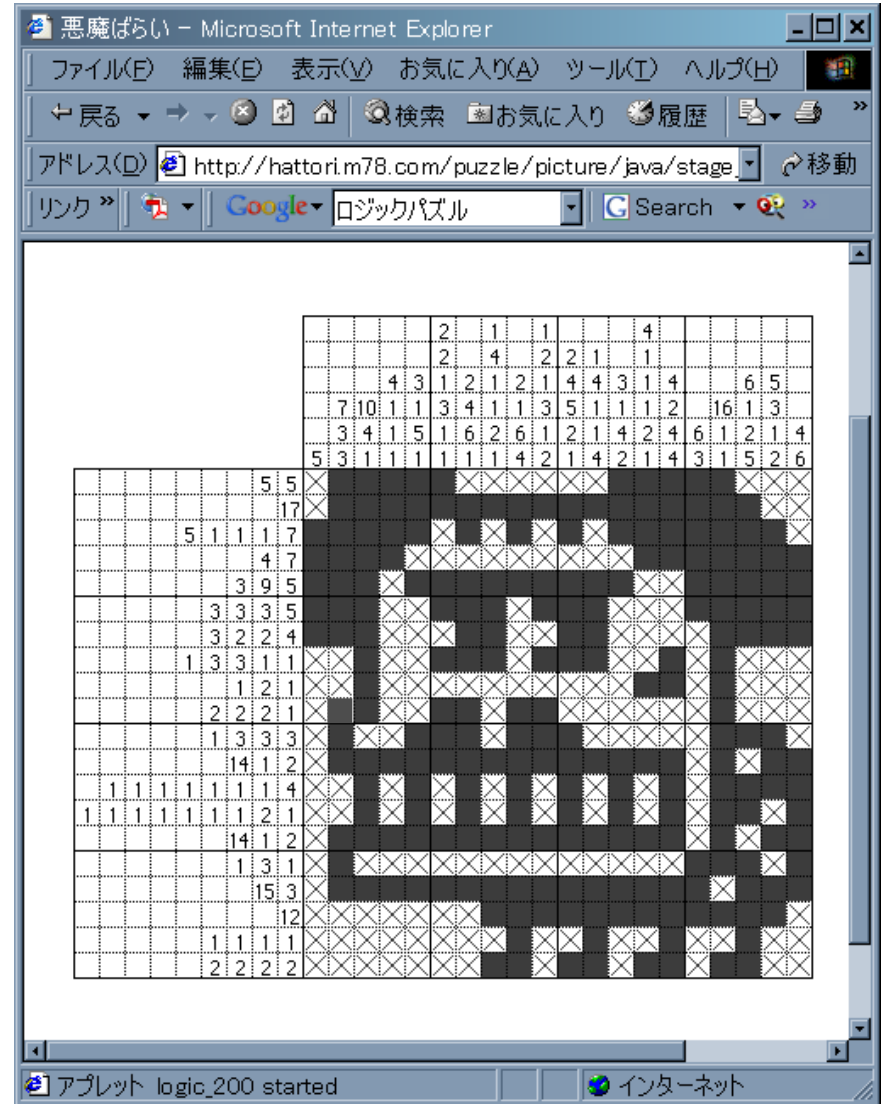
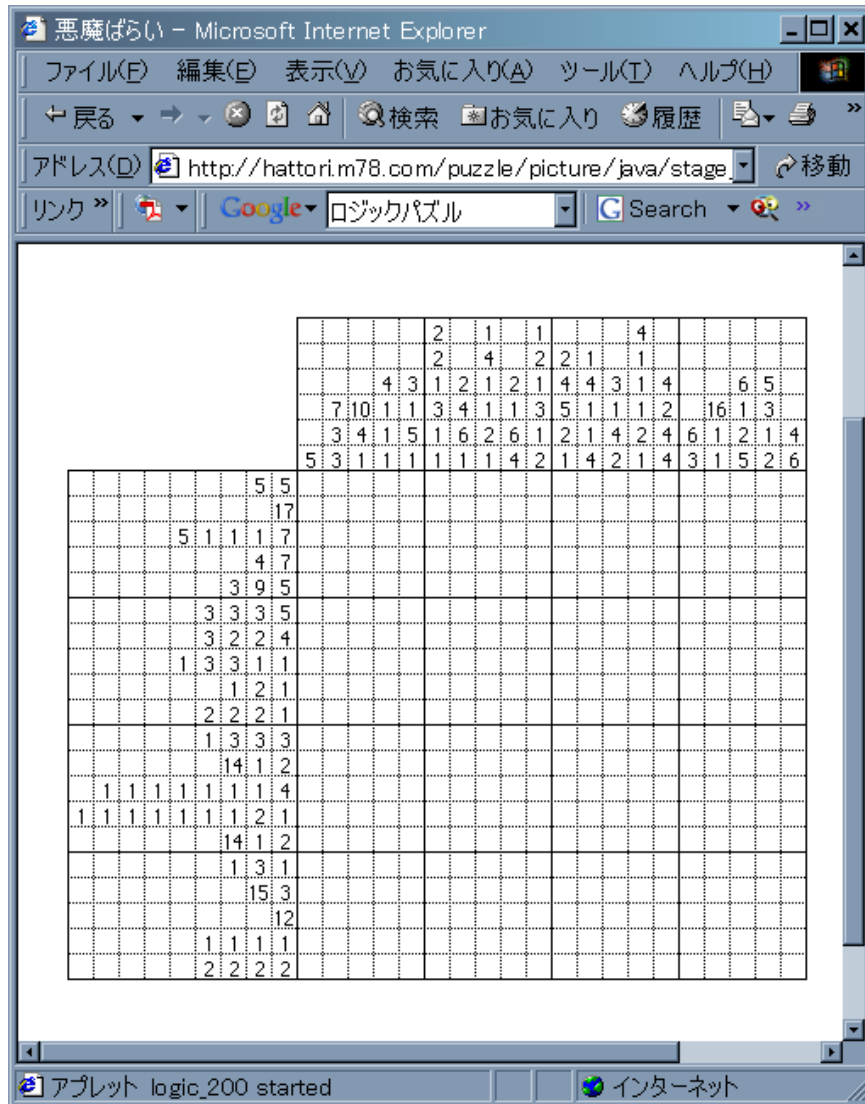
$\mathbf{x \text{ " * " } y = z}$

$z = \text{add}(x \wedge y_0, \text{add}(\rightarrow(x) \wedge y_1, \text{add}(\rightarrow^2(x) \wedge y_2, \dots)))$

$y_0 y_1 y_2 \dots y_n : n\text{-bits}$

More on encoding
~Logic Picture by SAT solver~

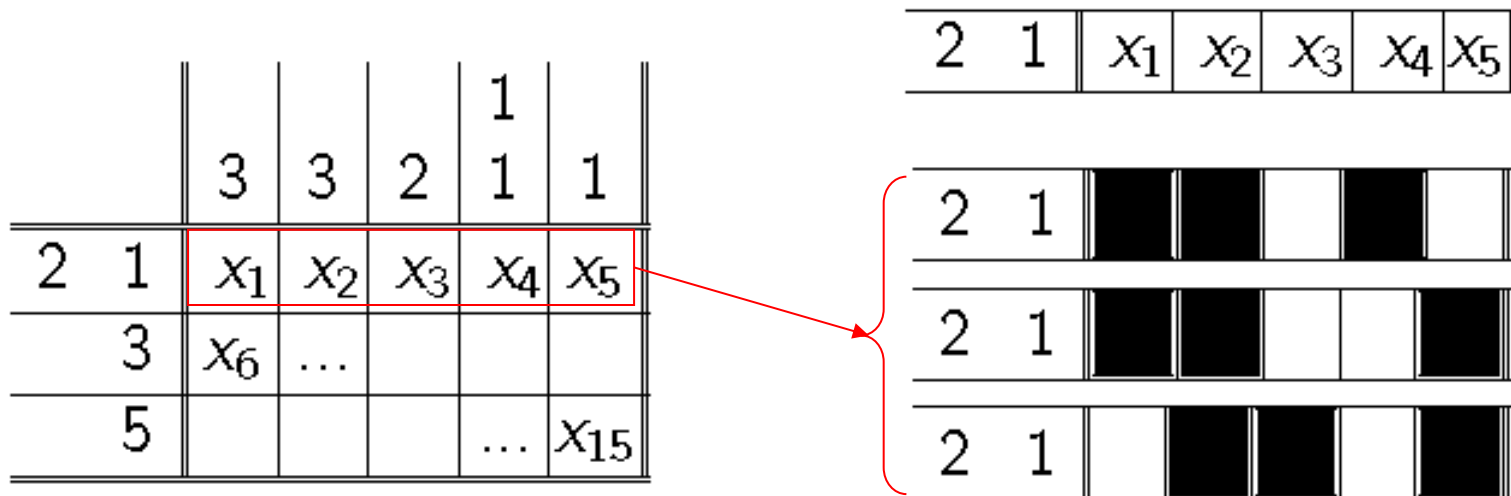
Logic pictures



p20x20_1

Logic picture encoding into SAT

- Assign a variable to each box, and encoding constraints on each column/row



$$(x_1 \wedge x_2 \wedge \neg x_3 \wedge x_4 \wedge \neg x_5)$$

$$\vee (x_1 \wedge x_2 \wedge \neg x_3 \wedge \neg x_4 \wedge x_5)$$

$$\vee (\neg x_1 \wedge x_2 \wedge x_3 \wedge \neg x_4 \wedge x_5)$$

- DNF needs to be converted into CNF

How to convert DNF into CNF ?

- Distribution law (logically equivalent CNF from DNF)

- ✓ Exponential growth and very inefficient

- $(x_1 \wedge x_2 \wedge \neg x_3 \wedge x_4 \wedge \neg x_5)$

- $\vee (x_1 \wedge x_2 \wedge \neg x_3 \wedge \neg x_4 \wedge x_5)$

- $\vee (\neg x_1 \wedge x_2 \wedge x_3 \wedge \neg x_4 \wedge x_5)$ becomes

- $(x_1 \vee x_1 \vee \neg x_1) \wedge \dots \wedge (x_1 \vee x_1 \vee x_5) \wedge$

- $(x_1 \vee x_2 \vee \neg x_1) \wedge \dots \wedge (x_1 \vee x_2 \vee x_5) \wedge \dots (5^3 \text{ clauses})$

- Tseitin conversion

- ✓ Obtain CNF of equivalent satisfiability with DNF

- ✓ Linear growth (for m-SAT, n-clauses, $\approx (m+1) n$ clauses), e.g., 19 clauses

- ✓ CNF in 3.8MB and solved in 0.6sec by miniSAT.

Tseitin conversion (1970)

- **Ex.** $(x_1 \wedge x_2 \wedge \neg x_3 \wedge x_4 \wedge \neg x_5)$

$\vee(x_1 \wedge x_2 \wedge \neg x_3 \wedge \neg x_4 \wedge x_5) \vee(\neg x_1 \wedge x_2 \wedge x_3 \wedge \neg x_4 \wedge x_5)$ is converted by

✓ Introduce fresh y_1, y_2, y_3 for $x_1 \wedge x_2 \wedge \neg x_3 \wedge x_4 \wedge \neg x_5$,

$x_1 \wedge x_2 \wedge \neg x_3 \wedge \neg x_4 \wedge x_5$, $\neg x_1 \wedge x_2 \wedge x_3 \wedge \neg x_4 \wedge x_5$

✓ $(y_1 \vee y_2 \vee y_3)$ and $y_1 \Leftrightarrow x_1 \wedge x_2 \wedge \neg x_3 \wedge x_4 \wedge \neg x_5$

– $y_1 \supset (x_1 \wedge x_2 \wedge \neg x_3 \wedge x_4 \wedge \neg x_5) \rightarrow$
 $(\neg y_1 \vee x_1) \wedge (\neg y_1 \vee x_2) \wedge (\neg y_1 \vee \neg x_3) \wedge (\neg y_1 \vee x_4) \wedge (\neg y_1 \vee \neg x_5)$

– $(x_1 \wedge x_2 \wedge \neg x_3 \wedge x_4 \wedge \neg x_5) \supset y_1 \rightarrow$
 $\neg x_1 \vee \neg x_2 \vee x_3 \vee \neg x_4 \vee x_5 \vee y_1$

This rule can be ignored (since y_1 to be 0 is crucial)

Tseitin Conversion (Tseitin 1970)

- Logically equivalent conversion may explode exponentially.
- Tseitin conversion is
 - ✓ at most square growth.
 - ✓ recursively adding a new variable as a substitute of each conjunctive subproposition.
- **Th.** Satisfiability is preserved (though validity is **not**).

Tseitin conversion

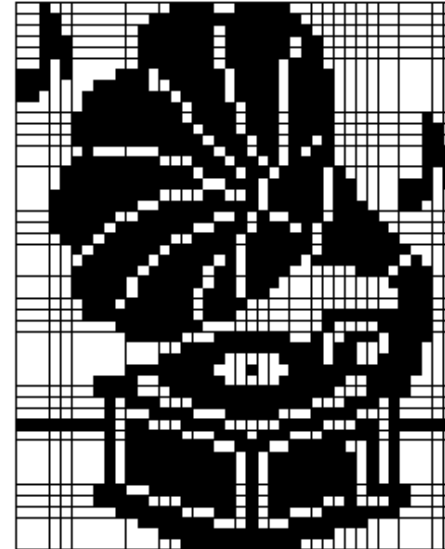
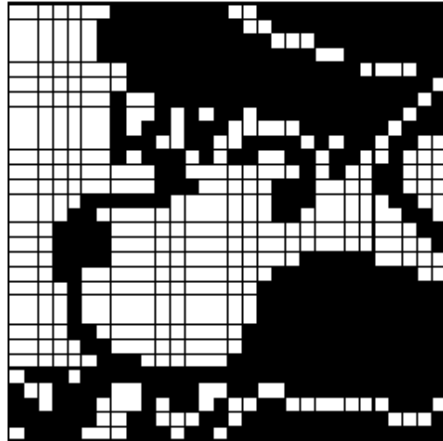
- **Th.** Each proposition A has a CNF A' (square to A in size), such that A is satisfiable if and only if A' is satisfiable.
- **Procedure.** Repeat the following procedure until reaching to a literal. (At each step, the number of literals decreases.)
 - ✓ Step 1: Replace $A \supset B$ with $\neg A \vee B$
 - ✓ Step 2: Push $\neg$ into innermost occurrences by $\neg(A \wedge B) \Leftrightarrow \neg A \vee \neg B$ and $\neg(A \vee B) \Leftrightarrow \neg A \wedge \neg B$.
 - ✓ Step 3: Remove double negation by $\neg \neg A \Leftrightarrow A$
 - ✓ Step 4a: If $A = B \wedge C$ (i.e., there exists $\wedge$ at the top level). Let B' and C' be CNFs with equal satisfiability to B and C , resp.
 - ✓ Step 4b: If $A = B \vee P$ for a propositional variable P . Let $B' = B'_1 \wedge \dots \wedge B'_m$ be a Tseitin encoding of B . Then, Tseitin encoding of A is $(B'_1 \vee P) \wedge \dots \wedge (B'_m \vee P)$.
 - ✓ Step 4c: If $A = B \vee C$ for a conjunctive proposition $C = C_1 \wedge \dots \wedge C_n$. Let B' be a Tseitin encoding of B and P a fresh propositional variable. Then, $B' \vee P$, $\neg P \vee C_1$, ..., $\neg P \vee C_n$, $\neg C_1 \vee \dots \vee \neg C_n \vee P$ are converted separately.

Large Logic Picture by SAT solver

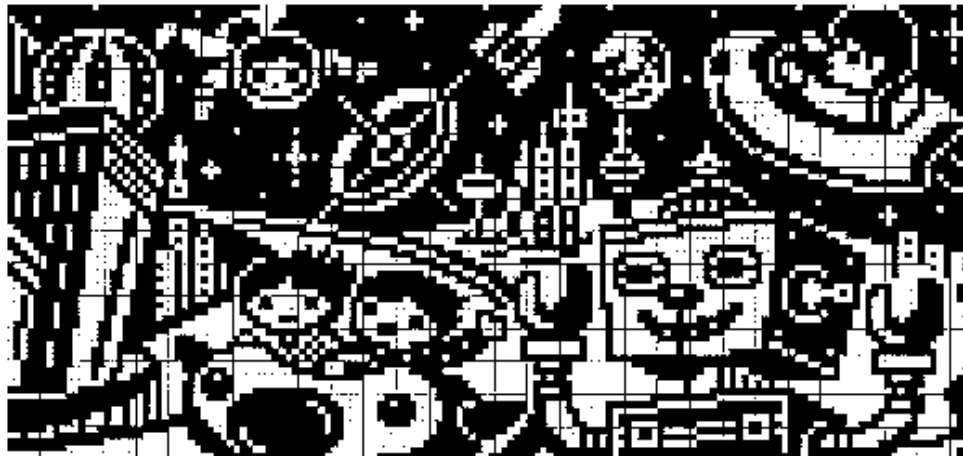
Tseitin conversion still not enough

- Logic pictures larger than 50×40 fail to generate CNF (at least $>1\text{GB}$)

30×30



50×40
p50x40_3

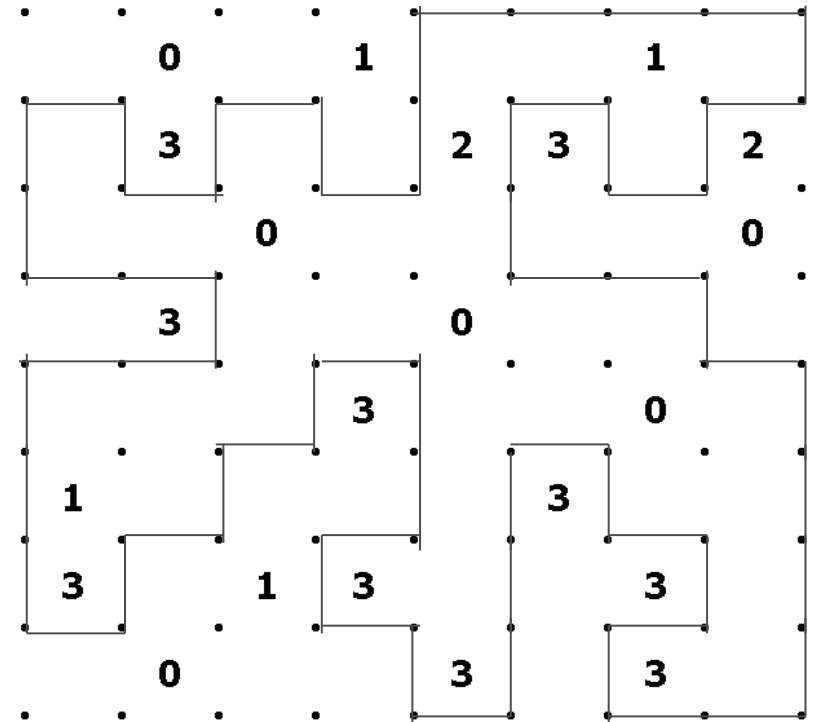


70×180

Mathematical Logic
(6) Practical SAT encoding
Slitherlink

Slitherlink

- Slitherlink
 - ✓ Find *a cycle* such that the numbers of edges of boxes match given numbers.
- Difficulties
 - ✓ How to describe “*Cycle is connected*”.
 - ✓ We consider directed edges.

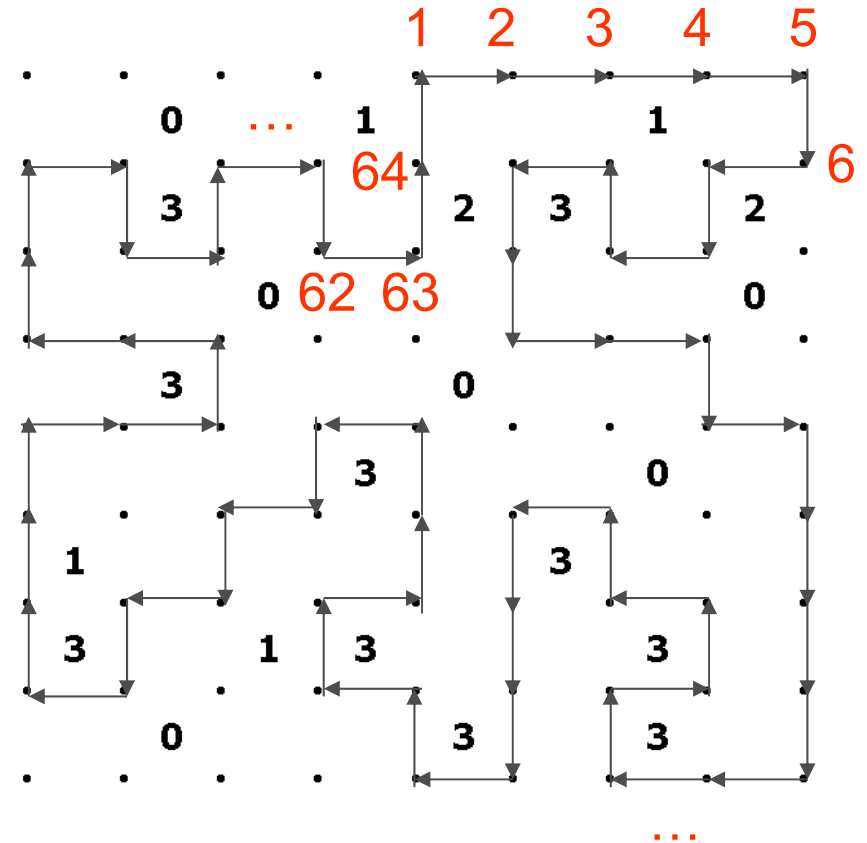


Difficulties

- Connectivity (or cycle) is not described in FOL (Gaiffman's Th.).
- In a directed (simple) graph, a standard definition is
 - ✓ $\text{closure}(X) = \forall x. \forall y. ((x \in X \wedge \text{edge}(x, y)) \supset y \in X)$
 - ✓ $\text{connected}(Z) = \forall X. \forall Y. (X \cup Y = Z \wedge X \cap Y = \emptyset \wedge \text{closure}(X) \wedge \text{closure}(Y)) \supset (X = \emptyset \vee Y = \emptyset)$
 - ✓ $\text{cycle}(X) = \text{connected}(X) \wedge \forall x \in X. \exists y \in X. \text{edge}(x, y) \wedge \forall x \in X. \exists y \in X. \text{edge}(y, x)$
- Our target is bounded.
 - ✓ Naïve solution: Enumerate all possibility in DNF.

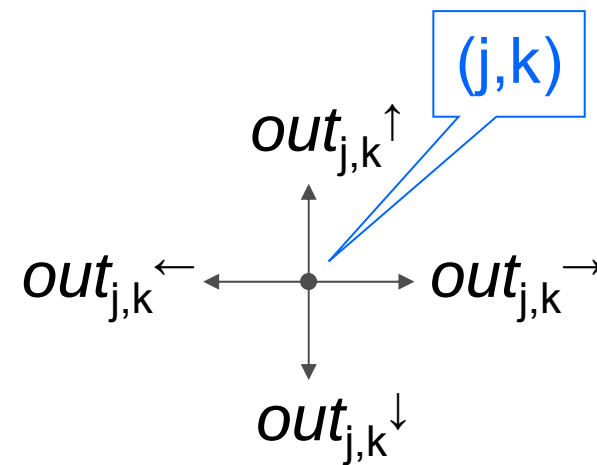
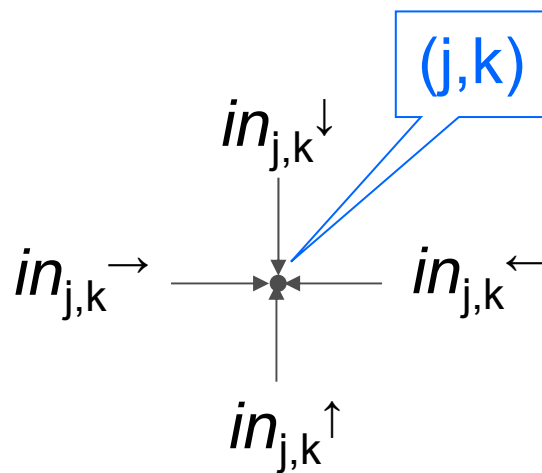
Different descriptions of a cycle

- X is a (directed) **cycle**
 - ✓ if X is **connected** and each vertex in X has unique successor and ancestor, respectively.
 - ✓ if any **increasing** label (starting from 1) on X has **a unique vertex labeled 1**, and
(Sugar, 神戸大学)

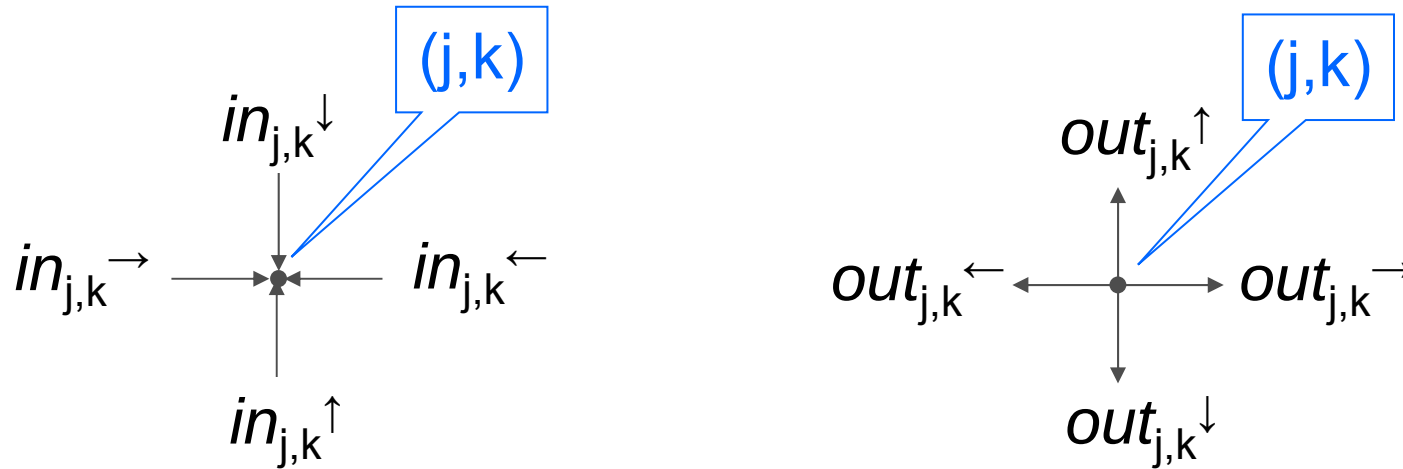


CNF description on a bounded model

- Variables for a vertex (j,k)
 - ✓ Incoming edges: $in_{j,k}^{\uparrow}$, $in_{j,k}^{\rightarrow}$, $in_{j,k}^{\downarrow}$, $in_{j,k}^{\leftarrow}$
 - ✓ Outgoing edges: $out_{j,k}^{\uparrow}$, $out_{j,k}^{\rightarrow}$, $out_{j,k}^{\downarrow}$, $out_{j,k}^{\leftarrow}$
 - ✓ Label (*int*) : $y_{j,k}$



Cycle constraints in CNF



- Unique incoming (outgoing) edge:
 $(in_{j,k}^{\uparrow} \vee in_{j,k}^{\rightarrow} \vee in_{j,k}^{\downarrow} \vee in_{j,k}^{\leftarrow}) \wedge (in_{j,k}^{\uparrow} \supset \neg in_{j,k}^{\rightarrow}) \wedge \dots$
- Incoming edge = outgoing edge of the predecessor.
 $in_{j,k}^{\rightarrow} = out_{j,k-1}^{\rightarrow}, in_{j,k}^{\rightarrow} \supset \neg out_{j,k}^{\leftarrow}, \dots$
- Increasing label: $in_{j,k}^{\rightarrow} \supset (y_{j,k+1} = 1 \vee y_{j,k+1} = y_{j,k} + 1), \dots$
- Unique vertex labeled 1: $(y_{j,k} = 1 \wedge y_{j',k'} = 1) \supset (j = j' \wedge k = k')$

Bit encoding of integers needed

Different definitions of a cycle

- (Directed) cycle definitions
 - ✓ $\text{cycle}(X) = \text{label!}(X) \wedge \forall x \in X. \exists y \in X. \text{edge}(x, y) \wedge \forall x \in X. \exists y \in X. \text{edge}(y, x)$
- We assume a labeling function $f: X \rightarrow \text{Nat}$
 - ✓ $\text{label}(f) = \forall x \in X. \forall y \in X. \text{edge}(x, y) \supset (f(y) = 1 \vee f(y) = f(x) + 1)$
 - ✓ $\text{label!}(X) = \forall f. \forall x \in X. \forall y \in X. (\text{label}(f) \wedge f(x) = 1 \wedge f(y) = 1) \supset x = y$
- Assume an upper bound n of the length of cycles, CNF label is described by $(x_{j,k,m} \wedge \text{in}_{j,k}^\uparrow) \supset (x_{j,k-1,m+1} \vee x_{j,k,0}), \dots, x_{j,k,0} \vee \dots \vee x_{j,k,n-1}, x_{j,k,0} \supset \neg x_{j,k,1}, \dots$ for $m < n$

Slitherlink specifications

- Box consists of 4 vertices $(j,k), (j+1,k), (j,k+1), (j+1,k+1)$
- The number of edges of (j,k) box is specified
 - ✓ $R_{j,k} = in_{j,k}^{\leftarrow} \vee out_{j,k}^{\rightarrow}, L_{j,k} = in_{j,k}^{\rightarrow} \vee out_{j,k}^{\leftarrow},$
 $U_{j,k} = in_{j,k}^{\downarrow} \vee out_{j,k}^{\uparrow}, D_{j,k} = in_{j,k}^{\uparrow} \vee out_{j,k}^{\downarrow}$
 - ✓ No 4 edges : $\neg L_{j,k} \vee \neg U_{j,k} \vee \neg R_{j,k} \vee \neg D_{j,k}$
 - ✓ 3 edges : $\neg L_{j,k} \supset (U_{j,k} \wedge R_{j,k} \wedge D_{j,k}), \dots$
 - ✓ 2 edges : $(\neg L_{j,k} \wedge \neg U_{j,k}) \Leftrightarrow (R_{j,k} \wedge D_{j,k}), \dots$
 - ✓ 1 edge : $L_{j,k} \vee U_{j,k} \vee R_{j,k} \vee D_{j,k},$
 $L_{j,k} \supset (\neg U_{j,k} \wedge \neg R_{j,k} \wedge \neg D_{j,k}), \dots$
 - ✓ No edges : $\neg L_{j,k} \wedge \neg U_{j,k} \wedge \neg R_{j,k} \wedge \neg D_{j,k}$